AI-ASSISTED CODING

ASSIGNMENT-11.3

NAME:V.SIDDARRTHA
HALL:2303a52062
BATCH:45

Task 1: Smart Contact Manager (Arrays & Linked Lists)
Scenario
SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.
Tasks
1. Implement the contact manager using arrays (lists).
2. Implement the same functionality using a linked list for dynamic memory allocation.
3. Implement the following operations in both approaches:
o Add a contact
o Search for a contact
o Delete a contact
4. Use GitHub Copilot to assist in generating search and delete methods.
5. Compare array vs. linked list approaches with respect to:
o Insertion efficiency
o Deletion efficiency
Expected Outcome
• Two working implementations (array-based and linked-list-based).
• A brief comparison explaining performance differences.

#PROMPT
Task 1: Smart Contact Manager (Arrays & Linked Lists)

Generate a Python program for a Contact Manager system.

Requirements:

1. Implement using an array (Python list) and a Linked List.

2. Each contact should contain name and phone number.

3. Provide operations:

- Add Contact

- Search Contact

- Delete Contact

- Display Contacts

4. Use classes and make the program menu-driven with user input.

5. Add comments explaining the code.

#code

```
class ContactManagerArray:
    def __init__(self):
        self.contacts = []

    def add_contact(self, name, phone):
        self.contacts.append({"name": name, "phone": phone})
        print("Contact added successfully")

    def search_contact(self, name):
        for contact in self.contacts:
            if contact["name"] == name:
                print("Contact Found:", contact)
                return
        print("Contact not found")

    def delete_contact(self, name):
        for contact in self.contacts:
            if contact["name"] == name:
                self.contacts.remove(contact)
                print("Contact deleted")
                return
        print("Contact not found")

    def display_contacts(self):
        print("\nContact List:")
        for contact in self.contacts:
            print(contact["name"], "-", contact["phone"])
```

```
cm = ContactManagerArray()

while True:
    print("\n1.Add 2.Search 3.Delete 4.Display 5.Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
        name = input("Enter name: ")
        phone = input("Enter phone: ")
        cm.add_contact(name, phone)

    elif choice == 2:
        name = input("Enter name to search: ")
        cm.search_contact(name)

    elif choice == 3:
        name = input("Enter name to delete: ")
        cm.delete_contact(name)

    elif choice == 4:
        cm.display_contacts()

    elif choice == 5:
        break

class Node:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.next = None
```

#output

```
PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/ass_11.3/task 1.py"

1.Add 2.Search 3.Delete 4.Display 5.Exit
Enter choice: 1
Enter name: siddhu
Enter phone: 9963959234
Contact added successfully

1.Add 2.Search 3.Delete 4.Display 5.Exit
Enter choice: 2
Enter name to search: siddartha
Contact not found

1.Add 2.Search 3.Delete 4.Display 5.Exit
Enter choice: 3
Enter name to delete: siddartha
Contact not found

1.Add 2.Search 3.Delete 4.Display 5.Exit
Enter choice:
```

```
powershell
Python
Python
Python
```

```python
class Node:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.next = None

class ContactManagerLinkedList:
    def __init__(self):
        self.head = None

    def add_contact(self, name, phone):
        new_node = Node(name, phone)
        new_node.next = self.head
        self.head = new_node
        print("Contact added")

    def search_contact(self, name):
        temp = self.head
        while temp:
            if temp.name == name:
                print("Found:", temp.name, "-", temp.phone)
                return
            temp = temp.next
        print("Contact not found")

    def delete_contact(self, name):
        temp = self.head
```

```
        prev = None

        while temp:
            if temp.name == name:
                if prev:
                    prev.next = temp.next
                else:
                    self.head = temp.next
                print("Contact deleted")
                return
            prev = temp
            temp = temp.next

        print("Contact not found")

    def display_contacts(self):
        temp = self.head
        while temp:
            print(temp.name, "-", temp.phone)
            temp = temp.next

cm = ContactManagerLinkedList()

while True:
    print("\n1.Add 2.Search 3.Delete 4.Display 5.Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
        name = input("Enter name: ")
        phone = input("Enter phone: ")
        cm.add_contact(name, phone)

    elif choice == 2:
        name = input("Enter name to search: ")
        cm.search_contact(name)

    elif choice == 3:
        name = input("Enter name to delete: ")
        cm.delete_contact(name)

    elif choice == 4:
        cm.display_contacts()

    elif choice == 5:
        break
```

#output

PROBLEMS DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Pyth
on314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/ass_11.3/task 1.py"

1.Add 2.Search 3.Delete 4.Display 5.Exit
Enter choice: 1
Enter name: siddartha
Enter phone: 9963959234
Contact added

1.Add 2.Search 3.Delete 4.Display 5.Exit
Enter choice: 2
Enter name to search: siddartha
Found: siddartha - 9963959234

1.Add 2.Search 3.Delete 4.Display 5.Exit
Enter choice: 3
Enter name to delete: siddartha
Contact deleted

1.Add 2.Search 3.Delete 4.Display 5.Exit
Enter choice:
```

Task 2: Library Book Search System (Queues & Priority Queues)
Scenario
The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.
Tasks
1. Implement a Queue (FIFO) to manage book requests.
2. Extend the system to a Priority Queue, prioritizing faculty requests.
3. Use GitHub Copilot to assist in generating:
o enqueue() method
o dequeue() method
4. Test the system with a mix of student and faculty requests.
Expected Outcome
• Working queue and priority queue implementations.
• Correct prioritization of faculty requests.

#prompt
Task 2: Library Book Search System (Queue & Priority Queue)
Generate a Python program for a Library Book Request System.

Requirements:
1. Implement a Queue (FIFO) using Python list.
2. Implement a Priority Queue where faculty requests are served before students.
3. Include methods:
  enqueue()
  dequeue()
  display()
4. Program should accept user input for requests.
Queue Implementation

#code

```
class Queue:
    def __init__(self):
        self.queue = []

    def enqueue(self, item):
        self.queue.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.queue.pop(0)
        else:
            return "Queue is empty"

    def is_empty(self):
        return len(self.queue) == 0

    def display(self):
        print("Queue:", self.queue)
# Priority Queue Implementation
class PriorityQueue:
    def __init__(self):
        self.faculty_queue = Queue()
        self.student_queue = Queue()

    def enqueue(self, item, priority):
        if priority == "faculty":
            self.faculty_queue.enqueue(item)
        elif priority == "student":
            self.student_queue.enqueue(item)
        else:
            print("Invalid priority. Use 'faculty' or 'student'.")

    def dequeue(self):
        if not self.faculty_queue.is_empty():
            return self.faculty_queue.dequeue()
        elif not self.student_queue.is_empty():
            return self.student_queue.dequeue()
        else:
            return "Both queues are empty"

    def display(self):
        print("Faculty Queue:")
        self.faculty_queue.display()
        print("Student Queue:")
        self.student_queue.display()
# Test the Library Book Request System
```

```python
if __name__ == "__main__":
    pq = PriorityQueue()

    while True:
        print("\n1. Enqueue Request")
        print("2. Dequeue Request")
        print("3. Display Queues")
        print("4. Exit")

        choice = input("Enter your choice: ")

        if choice == '1':
            name = input("Enter requester's name: ")
            priority = input("Enter priority (faculty/student): ")
            pq.enqueue(name, priority)

        elif choice == '2':
            print("Dequeued Request:", pq.dequeue())

        elif choice == '3':
            pq.display()

        elif choice == '4':
            break

        else:
            print("Invalid choice. Please try again.")
```

#code

PROBLEMS DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Pyth
on314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/ass_11.3/task 2.py"

1. Enqueue Request
2. Dequeue Request
3. Display Queues
4. Exit
Enter your choice: 1
Enter requester's name: siddartha
Enter priority (faculty/student): student

1. Enqueue Request
2. Dequeue Request
3. Display Queues
4. Exit
Enter your choice: 4
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

power
Pythor
Pythor
Pythor

Task 3: Emergency Help Desk (Stack Implementation)
Scenario
SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.
Tasks
1. Implement a Stack to manage support tickets.
2. Provide the following operations:
o push(ticket)
o pop()
o peek()
3. Simulate at least five tickets being raised and resolved.
4. Use GitHub Copilot to suggest additional stack operations such as:
o Checking whether the stack is empty
o Checking whether the stack is full (if applicable)
Expected Outcome
• Functional stack-based ticket management system.
• Clear demonstration of LIFO behavior.

#prompt
Task 3: Emergency Help Desk (Stack)
AI Prompt
Generate a Python program implementing a Stack for IT Help Desk tickets.

Requirements:
1. Implement push(), pop(), peek()
2. Include methods to check stack empty and full.
3. Use list to implement stack.
4. Simulate raising and resolving tickets.
5. Use menu driven input.

#code
```
class Stack:
    def __init__(self, capacity):
        self.stack = []
        self.capacity = capacity

    def push(self, item):
        if len(self.stack) < self.capacity:
            self.stack.append(item)
            print("Ticket raised:", item)
        else:
            print("Stack is full. Cannot raise more tickets.")

    def pop(self):
        if not self.is_empty():
            resolved_ticket = self.stack.pop()
```

```
            print("Ticket resolved:", resolved_ticket)
        else:
            print("Stack is empty. No tickets to resolve.")

    def peek(self):
        if not self.is_empty():
            return self.stack[-1]
        else:
            return "Stack is empty."

    def is_empty(self):
        return len(self.stack) == 0

    def is_full(self):
        return len(self.stack) == self.capacity
# Test the Emergency Help Desk Stack
if __name__ == "__main__":
    help_desk = Stack(capacity=5)

    while True:
        print("\n1.Raise Ticket 2.Resolve Ticket 3.Peek Next Ticket 4.Exit")
        choice = int(input("Enter choice: "))

        if choice == 1:
            ticket = input("Enter ticket description: ")
            help_desk.push(ticket)

        elif choice == 2:
            help_desk.pop()

        elif choice == 3:
            next_ticket = help_desk.peek()
            print("Next ticket to resolve:", next_ticket)

        elif choice == 4:
            break

        else:
            print("Invalid choice. Please try again.")
```

#output

```
PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

1.Raise Ticket 2.Resolve Ticket 3.Peek Next Ticket 4.Exit
Enter choice: 1
Enter ticket description: train
Ticket raised: train

1.Raise Ticket 2.Resolve Ticket 3.Peek Next Ticket 4.Exit
Enter choice: 
```

Task 4: Hash Table
Objective
To implement a Hash Table and understand collision handling.
Task Description
Use AI to generate a hash table with:
- **Insert**
- **Search**
- **Delete**

Starter Code
class HashTable:
pass
Expected Outcome
- **Collision handling using chaining**
- **Well-commented methods**

#prompt
Task 4: Hash Table
AI Prompt
Generate a Python Hash Table implementation with collision handling using chaining.

Requirements:
1. Methods:
   insert(key,value)
   search(key)
   delete(key)
2. Use lists for chaining.
3. Add comments explaining hash function and collisions.

Hash Table Code

#code
```
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]  # Create a list of empty lists for chaining
```

```
    def hash_function(self, key):
        """Simple hash function to compute index for a given key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert a key-value pair into the hash table."""
        index = self.hash_function(key)
        # Check if the key already exists and update it
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)  # Update existing key
                return
        # If key does not exist, add new key-value pair
        self.table[index].append((key, value))

    def search(self, key):
        """Search for a value by its key in the hash table."""
        index = self.hash_function(key)
        for k, v in self.table[index]:
            if k == key:
                return v  # Return the value if key is found
        return None  # Return None if key is not found

    def delete(self, key):
        """Delete a key-value pair from the hash table."""
        index = self.hash_function(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]  # Remove the key-value pair
                return True  # Return True if deletion is successful
        return False  # Return False if key is not found
# Test the Hash Table
if __name__ == "__main__":
    hash_table = HashTable()

    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    hash_table.insert("city", "New York")

    print("Search for 'name':", hash_table.search("name"))  # Output: Alice
    print("Search for 'age':", hash_table.search("age"))    # Output: 30
    print("Search for 'city':", hash_table.search("city"))  # Output: New York

    hash_table.delete("age")
    print("Search for 'age' after deletion:", hash_table.search("age"))  # Output: None
```

#output

```
PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Pyth
on314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/ass_11.3/task 4.py"
Search for 'name': Alice
Search for 'age': 30
Search for 'city': New York
Search for 'age' after deletion: None
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Task 5: Real-Time Application Challenge
Scenario
Design a Campus Resource Management System with the following features:
- **Student Attendance Tracking**
- **Event Registration System**
- **Library Book Borrowing**
- **Bus Scheduling System**
- **Cafeteria Order Queue**

Student Tasks
1. Choose the most appropriate data structure for each feature.
2. Justify your choice in 2–3 sentences.
3. Implement one selected feature using AI-assisted code generation.
Expected Outcome
- **Mapping table: Feature → Data Structure → Justification**
- **One fully working Python implementation**

#prompt
Task 5: Real-Time Application Challenge
Mapping Table

Feature	Data Structure	Justification
Attendance Tracking	Hash Table	Fast lookup for student IDs
Event Registration	Linked List	Dynamic addition of participants
Library Borrowing	Queue	First come first served
Bus Scheduling	Graph	Stops connected as routes
Cafeteria Orders	Queue	Orders processed sequentially

AI Prompt
Create a Python program for a Cafeteria Order Queue system.

Requirements:
1. Use Queue data structure.
2. Allow placing order and serving order.
3. Use menu driven program.
4. Display current orders.

#code

```
class CafeteriaOrderQueue:
    def __init__(self):
        self.orders = []

    def place_order(self, order):
        self.orders.append(order)
        print("Order placed:", order)

    def serve_order(self):
        if not self.is_empty():
            served_order = self.orders.pop(0)
            print("Order served:", served_order)
        else:
            print("No orders to serve.")

    def is_empty(self):
        return len(self.orders) == 0

    def display_orders(self):
        print("Current Orders:")
        for order in self.orders:
            print(order)
# Test the Cafeteria Order Queue
if __name__ == "__main__":
    cafeteria_queue = CafeteriaOrderQueue()

    while True:
        print("\n1.Place Order 2.Serve Order 3.Display Orders 4.Exit")
        choice = int(input("Enter choice: "))

        if choice == 1:
            order = input("Enter order details: ")
            cafeteria_queue.place_order(order)

        elif choice == 2:
            cafeteria_queue.serve_order()

        elif choice == 3:
            cafeteria_queue.display_orders()

        elif choice == 4:
            print("Exiting the program.")
            break
```

#output

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Py
on314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/ass_11.3/task 5.py"

1.Place Order 2.Serve Order 3.Display Orders 4.Exit
Enter choice: 1
Enter order details: laptop
Order placed: laptop

1.Place Order 2.Serve Order 3.Display Orders 4.Exit
Enter choice:
```